



CSS Variables (Custom Properties)

CSS variables let you store values in your stylesheet and reuse them anywhere. The syntax has two parts: defining and consuming.

```
:root {  
  --primary: #3b82f6;  
  --spacing: 8px;  
  --radius: 6px;  
}  
  
.button {  
  background: var(--primary);  
  padding: var(--spacing);  
  border-radius: var(--radius);  
}
```

Names always start with `--`. You read them with `var(--name)`, optionally with a fallback: `var(--name, gray)`. You'll usually define them on `:root` (which is the `<html>` element), but you can scope them anywhere — and that's where they get powerful.

Scoping and inheritance. CSS variables cascade and inherit like normal properties. Define one on a parent, and every descendant can read it. Redefine it on a child, and the child (and its subtree) sees the new value.

```
.card {  
  --bg: white;  
  background: var(--bg);  
}
```

```
.card.dark {  
  --bg: #1f2937; /* overrides only inside .card.dark */  
}
```

The classic use case — theming:

```
:root {  
  --bg: white;  
  --text: #111;  
}  
  
[data-theme="dark"] {  
  --bg: #111;  
  --text: #eee;  
}  
  
body {  
  background: var(--bg);  
  color: var(--text);  
}
```

Toggle the `data-theme` attribute on `<html>` and your entire site re-themes.

Math with `calc()`:

`::before` and `::after`

These pseudo-elements insert a virtual child at the very start (`::before`) or very end (`::after`) of an element's content. They're real, renderable boxes — you can size them, position them, animate them — but they don't exist in the DOM, so JavaScript can't grab them with `querySelector`.

The one non-negotiable rule: **they don't render without the `content` property.** Even an empty string counts.

```
.fancy::before {
  content: "";          /* required, even if empty */
  display: block;
  width: 40px;
  height: 2px;
  background: red;
}
```

Decorative shapes:

```
.section-divider::before {
  content: "";
  display: block;
  height: 1px;
  background: linear-gradient(to right, transparent, #888, transparent);
  margin: 2rem 0;
}
```

Reading attributes from HTML with `attr()` — great for tooltips:

```
<span class="tooltip" data-tip="Save your work">💾</span>
```

```
.tooltip { position: relative; }

.tooltip:hover::after {
  content: attr(data-tip);
  position: absolute;
  bottom: 100%;
  left: 50%;
  transform: translateX(-50%);
  background: #111;
  color: white;
  padding: 4px 8px;
```

```
border-radius: 4px;
white-space: nowrap;
}
```

Counters — automatically numbered content without touching markup:

```
ol.steps { counter-reset: step; list-style: none; }

ol.steps li::before {
  counter-increment: step;
  content: "Step " counter(step) ": ";
  font-weight: bold;
  color: #3b82f6;
}
```

Quotation marks around blockquotes:

```
blockquote::before { content: "\201C"; } /* " */
blockquote::after { content: "\201D"; } /* " */
```

What `content` can hold: strings (`"hello"`), attributes (`attr(href)`), counters (`counter(x)`), images (`url('icon.svg')`), or any combination of those: `content: "Page " counter(p) " of " counter(total);`.

Limitations. You cannot use `::before` / `::after` on *replaced elements* — that includes `<img>`, `<input>`, `<video>`, `<iframe>`, `<br>`, `<hr>`. The browser draws their content itself and has no "inside" for pseudo-elements to attach to. The common workaround for styling checkboxes is to hide the real input and style its `<label>` instead.

Syntax note. `:before` (single colon) is the old CSS2 syntax and still works for backwards compatibility. `::before` (double colon) is the modern CSS3 standard — use that one. Pseudo-elements are inline by default; set `display: block` or `inline-block` if you want to give them dimensions.

Tailwind CSS

Tailwind is a **utility-first** CSS framework. Instead of writing custom classes (`.card` , `.btn-primary`), you compose styles from tiny single-purpose utility classes directly in your HTML.

```
<button class="bg-blue-500 hover:bg-blue-700 text-white font-  
bold py-2 px-4 rounded">  
  Click me  
</button>
```

Each class maps to one or two CSS declarations: `bg-blue-500` → `background-color: #3b82f6` , `py-2` → `padding-top: 0.5rem; padding-bottom: 0.5rem` , etc.

The naming patterns to internalize:

- **Spacing:** `m-4` margin, `p-4` padding, with directional variants `mt-` / `mr-` / `mb-` / `ml-` and axis variants `mx-` / `my-` . The number is on a scale where `1 = 0.25rem (4px)` , so `m-4 = 1rem` .
- **Sizing:** `w-64` , `h-screen` , `max-w-md` , `min-h-0` . Same scale as spacing, plus keywords (`full` , `screen` , `auto`).
- **Colors:** `bg-{color}-{shade}` , `text-{color}-{shade}` , `border-{color}-{shade}` . Shades run 50, 100, 200, ..., 900, 950.
- **Typography:** `text-sm` , `text-xl` , `font-bold` , `tracking-wide` (letter-spacing), `leading-tight` (line-height).
- **Flexbox:** `flex` , `flex-col` , `items-center` , `justify-between` , `gap-4` .
- **Grid:** `grid grid-cols-3 gap-4` .

Responsive design — mobile-first prefixes:

```
<div class="w-full md:w-1/2 lg:w-1/3">  
  Full width on phones, half on tablets, third on desktops.  
</div>
```

Breakpoints: `sm:` (≥640px), `md:` (≥768px), `lg:` (≥1024px), `xl:` (≥1280px), `2xl:` (≥1536px). They're min-width queries, so styles cascade upward — write the mobile version unprefix, then override at each larger breakpoint.

State variants — the most useful ones:

```
<button class="
  bg-blue-500
  hover:bg-blue-700
  focus:ring-2 focus:ring-blue-300
  active:bg-blue-800
  disabled:opacity-50 disabled:cursor-not-allowed
">
```

Others worth knowing: `dark:`, `group-hover:` (when a parent with class `group` is hovered), `peer-checked:` (style based on a sibling's state), `first:`, `last:`, `odd:`, `even:`, and arbitrary attribute selectors like `aria-expanded:` and `data-[state=open]:`.

Dark mode:

```
<div class="bg-white text-black dark:bg-gray-900 dark:text-wh
ite">
```

Toggle by either setting a `dark` class on `<html>` (with `darkMode: 'class'` in config) or letting it follow the OS preference (`darkMode: 'media'`).

before: and **after:** **variants** (full-circle moment) — Tailwind has utilities for pseudo-elements:

```
<span class="
  relative
  before:content-['→']
  before:mr-2
  before:text-blue-500
">
  Next step
</span>
```

Note `before:content-['→']` — you have to set content explicitly, just like in raw CSS.

Arbitrary values — the escape hatch for one-off values not in the default scale:

```
<div class="w-[347px] bg-[#ff6b6b] grid-cols-[1fr_2fr_1fr] to
p-[calc(100%+8px)]">
```

Use these for true one-offs; reach for the config for repeated values.

Customizing — `tailwind.config.js` (v3) or `@theme` (v4):

```
// v3
module.exports = {
  theme: {
    extend: {
      colors: { brand: '#3b82f6' },
      fontFamily: { display: ['Inter', 'sans-serif'] },
    },
  },
}
```

```
/* v4 */
@theme {
  --color-brand: #3b82f6;
  --font-display: 'Inter', sans-serif;
}
```

Then `bg-brand` and `font-display` just work — and notice v4 uses CSS variables under the hood, which is the bridge between everything you've just learned.

@apply — extracting repeated patterns into a single class:

```
@layer components {
  .btn-primary {
    @apply bg-blue-500 text-white px-4 py-2 rounded hover:bg-
    blue-700;
  }
}
```

Use this sparingly. Tailwind's whole pitch is composition in markup; if you're `@apply`-ing everything, you're just writing regular CSS with extra steps. Prefer extracting a *component* (React, Vue, Svelte) over extracting a *class*.

Pros: rapid iteration, no naming-things-fatigue, consistent design tokens, tiny production bundle (the JIT compiler only ships utilities you actually used).

Cons: HTML gets verbose, real learning curve for the class names, less semantic markup. The verbosity problem gets solved naturally once you componentize.
